

Computer Organisation

Topic 6

Addressing Modes and Instruction Formats

Hamacher et al., Computer Organisation (5th Ed.), Chap. 2

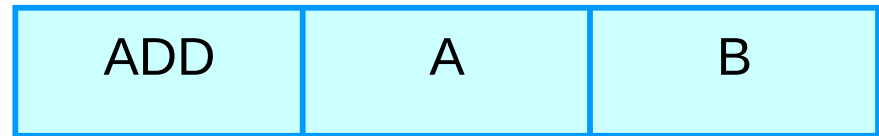
Stallings, Computer Organization & Architecture (9th Ed.), Chap. 13

Outline

- Addressing modes
 - Immediate
 - Direct
 - Indirect
 - Register
 - Register Indirect
 - Displacement
- Instruction Formats

Addressing Modes

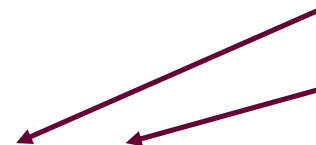
- To achieve this, various **addressing modes** are employed
- Trade-off
 - Address **range**/addressing **flexibility**
 - Number of memory **references**/complexity of **address calculation**



6 bits

8 bits

8 bits

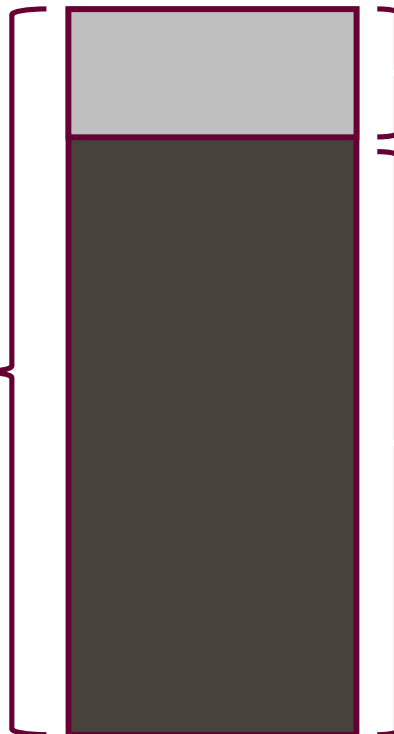


Memory

256 locations

4G locations

??



Rough Analogy

- Address
 - 401/G31
 - School of Computer Science
 - Universiti Sains Malaysia

Rough Analogy

Immediate Addressing

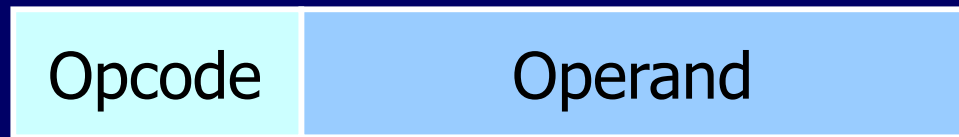
- Operand is **part of instruction**
- Operand = **address field**
- e.g. ADD 5
 - Add 5 (the operand) to contents of accumulator

Immediate Addressing

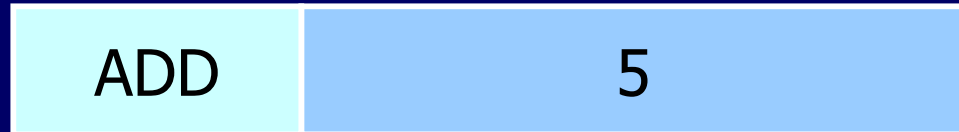
- **No memory reference** to fetch data (pro)
- **Fast** (pro)
- **Limited range** (con) - limited to the size of the address field
- **Data is constant** at run time

Immediate Addressing Diagram

Instruction



e.g.



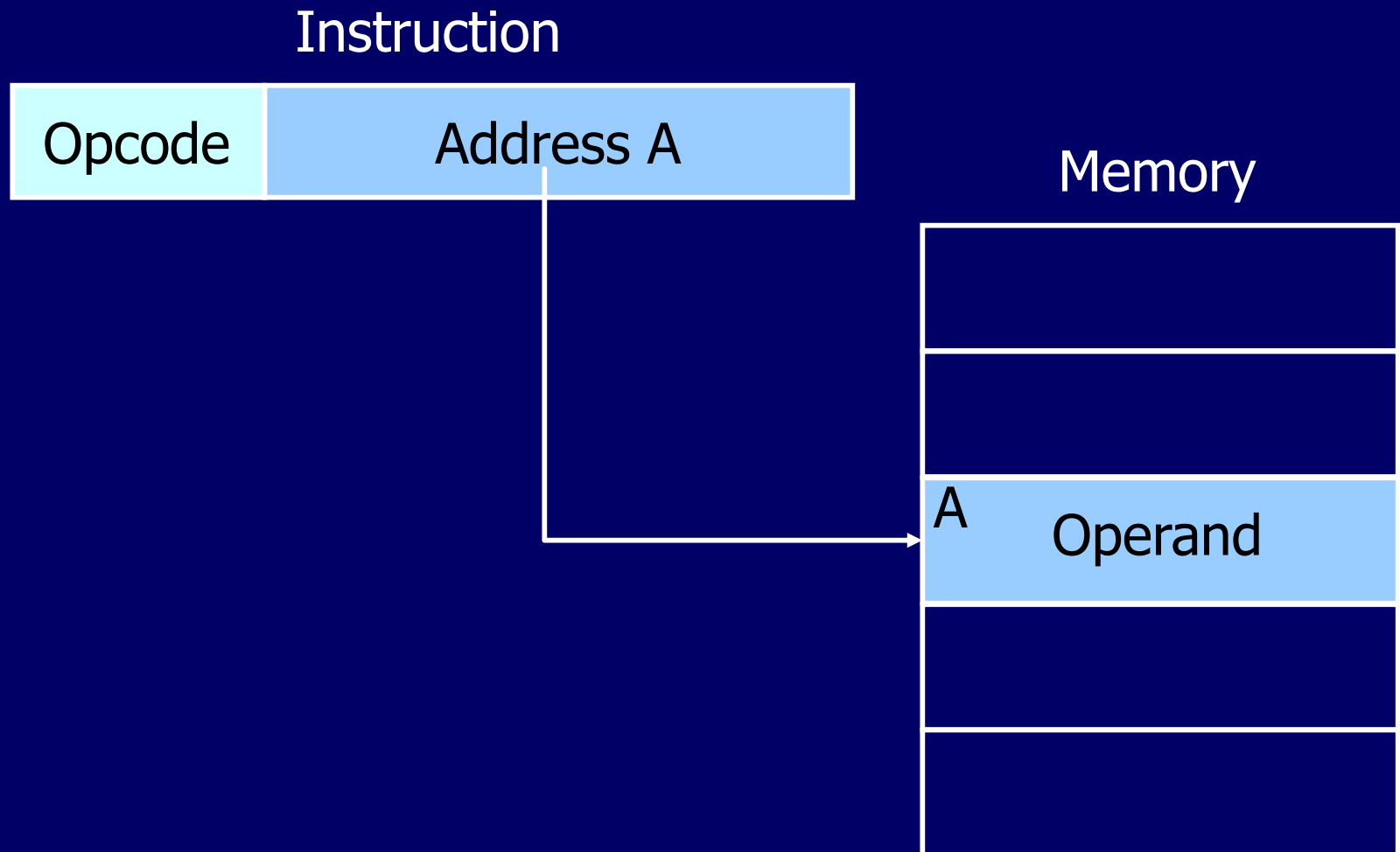
Direct Addressing

- Address field contains **address of operand**
- Effective address (EA) = **address field (A)**
- e.g. ADD A
 - Add contents of cell A to accumulator
 - Look in memory at address A for operand

Direct Addressing

- **Single memory reference** to access data (pro)
- **No additional calculations** to work out effective address (pro)
- **Limited address space** (con)
- Constant address. **Data may change** at run time

Direct Addressing Diagram



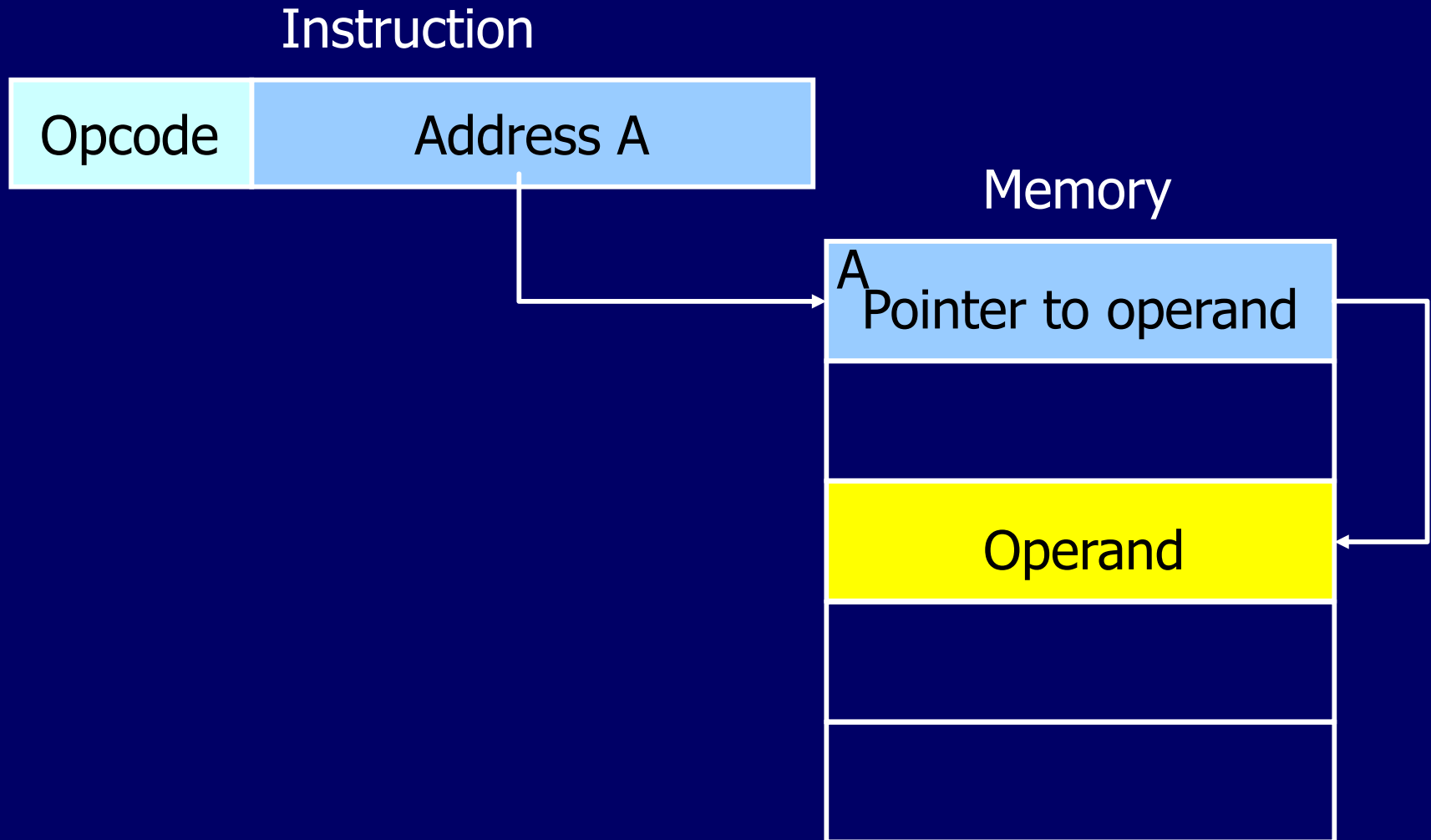
Indirect Addressing

- Memory cell pointed to by address field **contains the address** of (pointer to) the operand
- **2 memory access** required:
 - one for effective address
 - another for operand

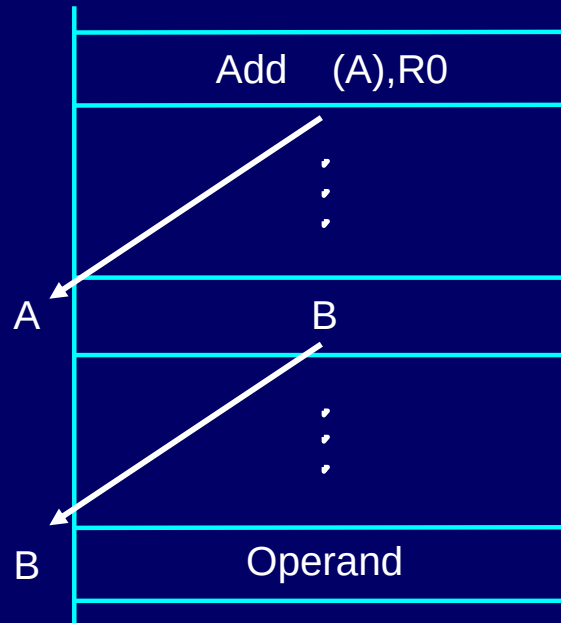
Indirect Addressing

- $EA = (A)$
 - Look in A, find address (A) and look there for operand
- e.g. ADD (A)
 - Add contents of cell pointed to by contents of A to accumulator

Indirect Addressing Diagram



Indirect Addressing Diagram



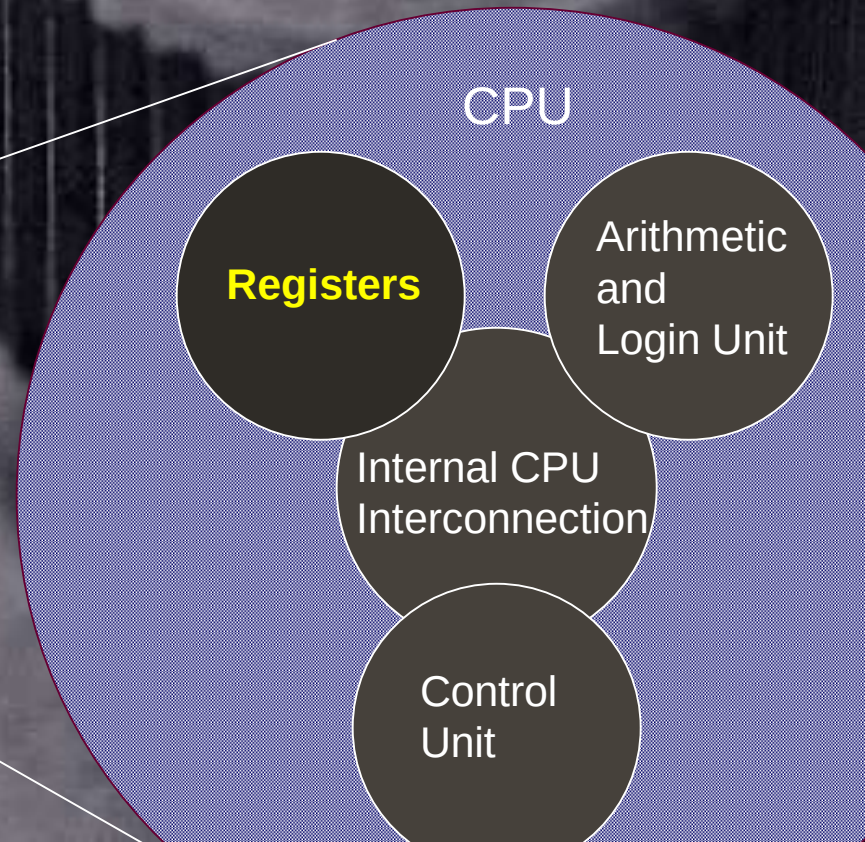
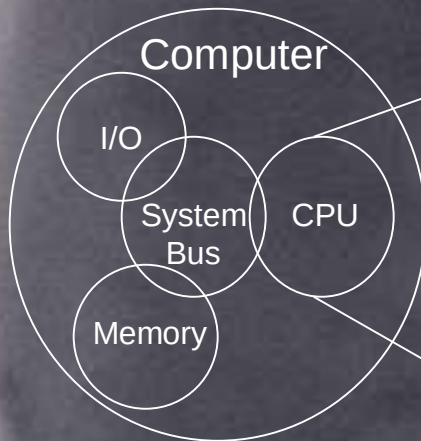
Indirect Addressing

- **Large address space** (pro)
- Range of **effective address is 2^n**
where n = memory word length
- Number of **locations** to hold effective address constrained to **2^k** where k = width of instruction's address field

Indirect Addressing

- May be **nested**, multilevel, cascaded
 - $EA = (\dots (A) \dots)$
- **Multiple memory accesses** to find operand (con)
- Hence **slower** (con)

Structure The CPU



Register Addressing

- Similar to **Direct Addressing**
- **Operand is held in register** named in address field
- $EA = R$ [address of the register]
- Limited number of registers
 - Smaller range of address

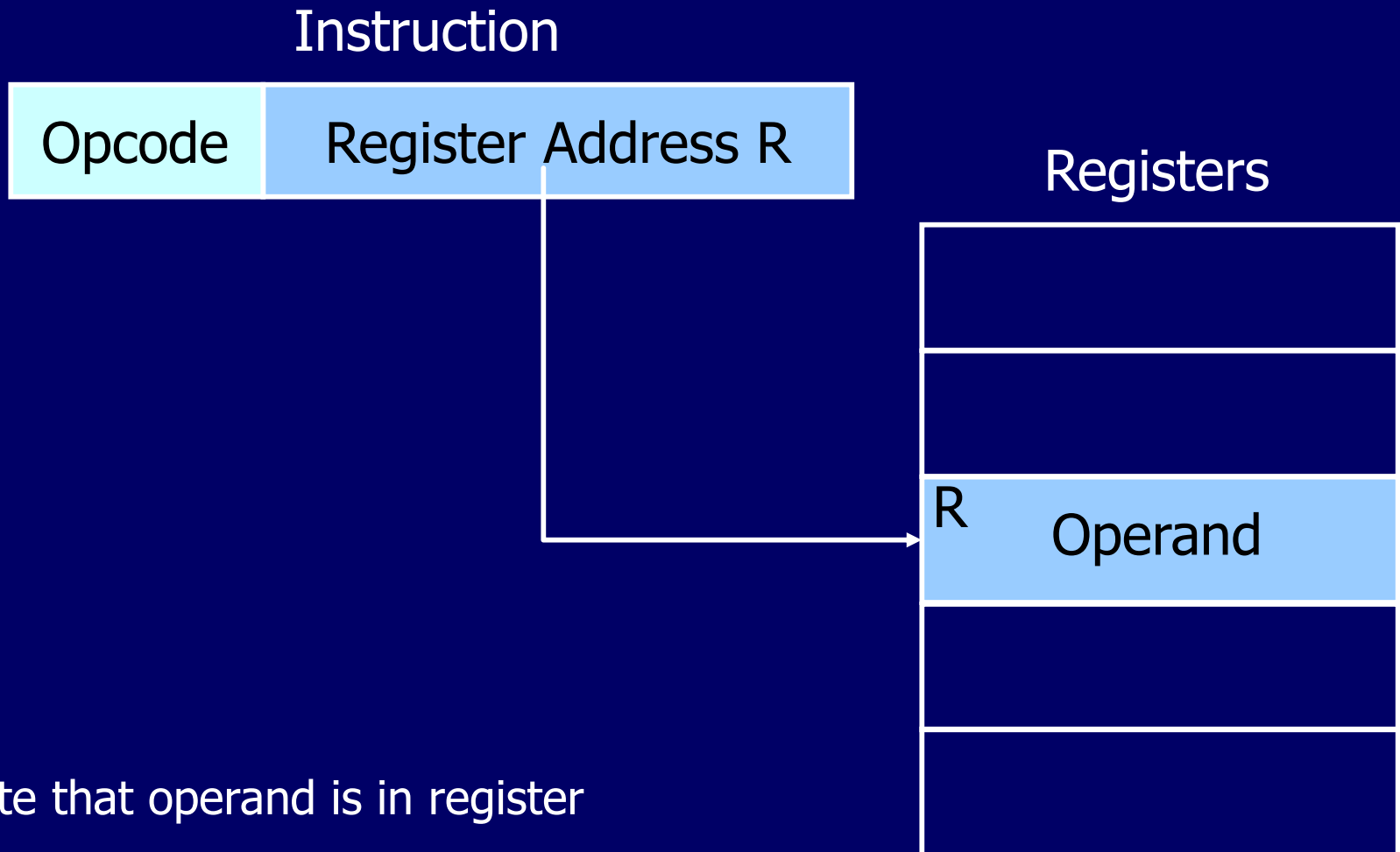
Register Addressing

- Very **small address field** needed (pro)
 - Shorter instructions
 - Faster instruction fetch
- **No memory access** (pro)
- Very **fast** execution (pro)
- Very **limited address space** (con)

Register Addressing

- Multiple registers helps performance
 - Especially when operand in register remains in use for multiple operations
 - Requires good assembly programming or compiler writing
 - N.B. C programming
 - register int a;

Register Addressing Diagram



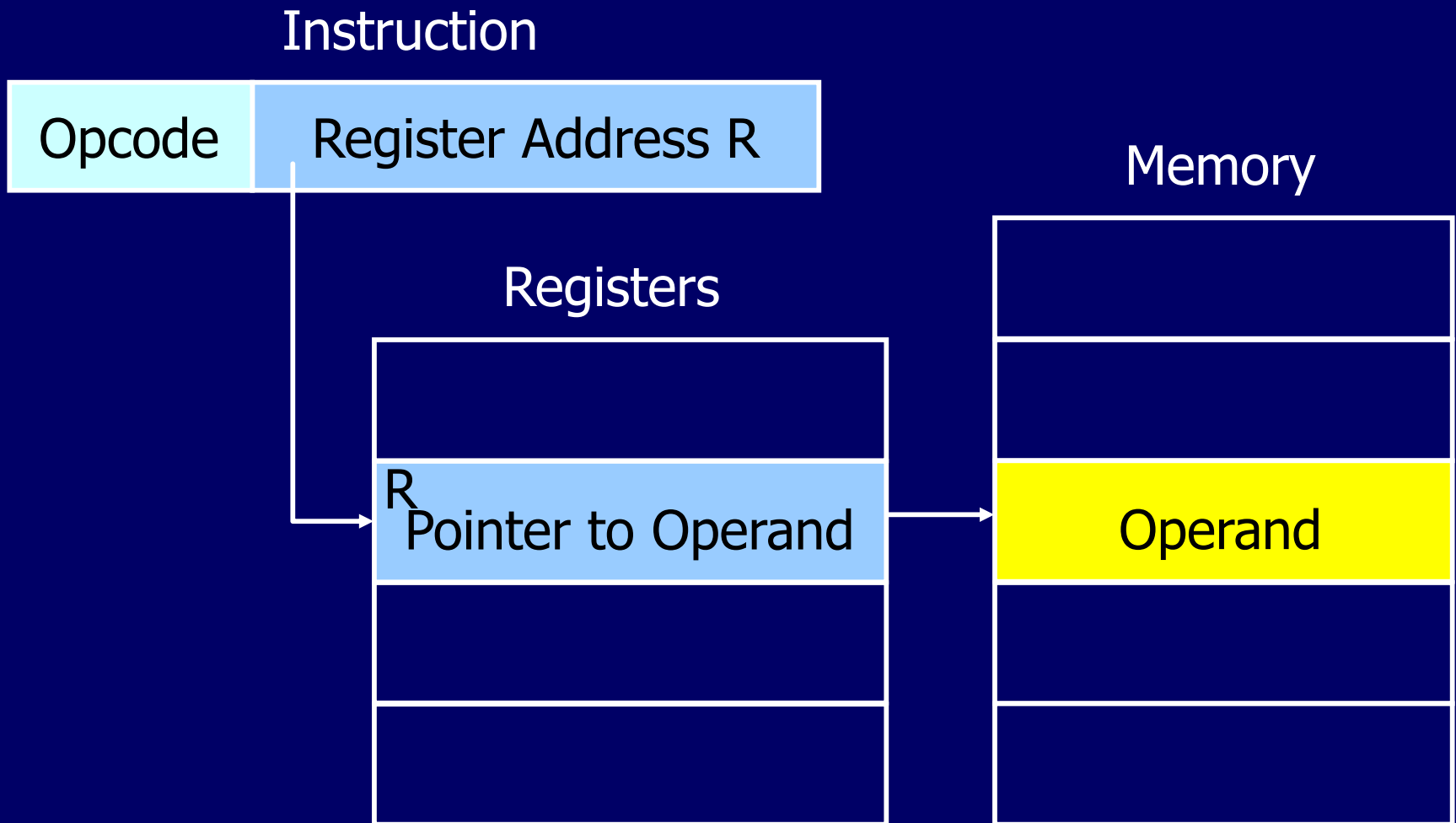
Register Indirect Addressing

- Similar to **indirect addressing**
- $EA = (R)$
- Operand is in memory cell pointed to by contents of register R

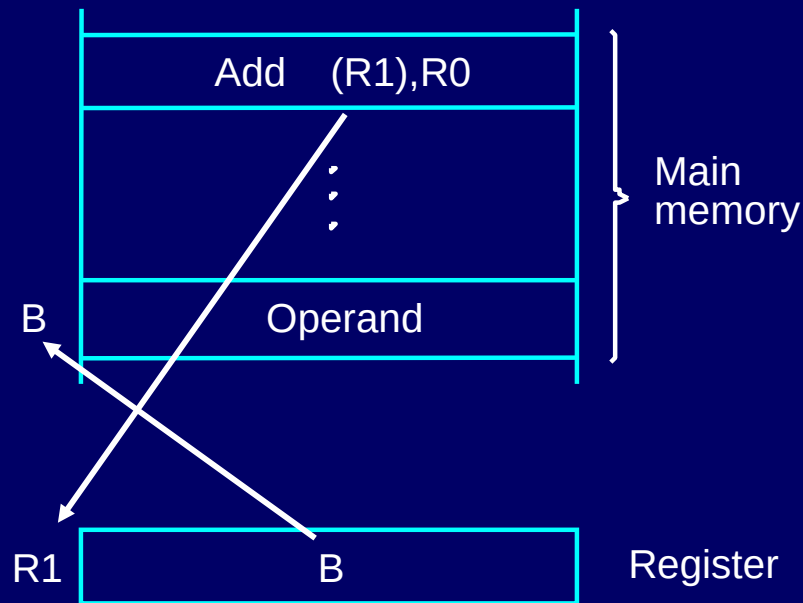
Register Indirect Addressing

- **Large address space** (2^n) (pro)
- **Multiple memory accesses** to find operand (con)
- But **one fewer memory access** than indirect addressing (i.e. one register and one memory access)

Register Indirect Addressing Diagram



Register Indirect Addressing Diagram



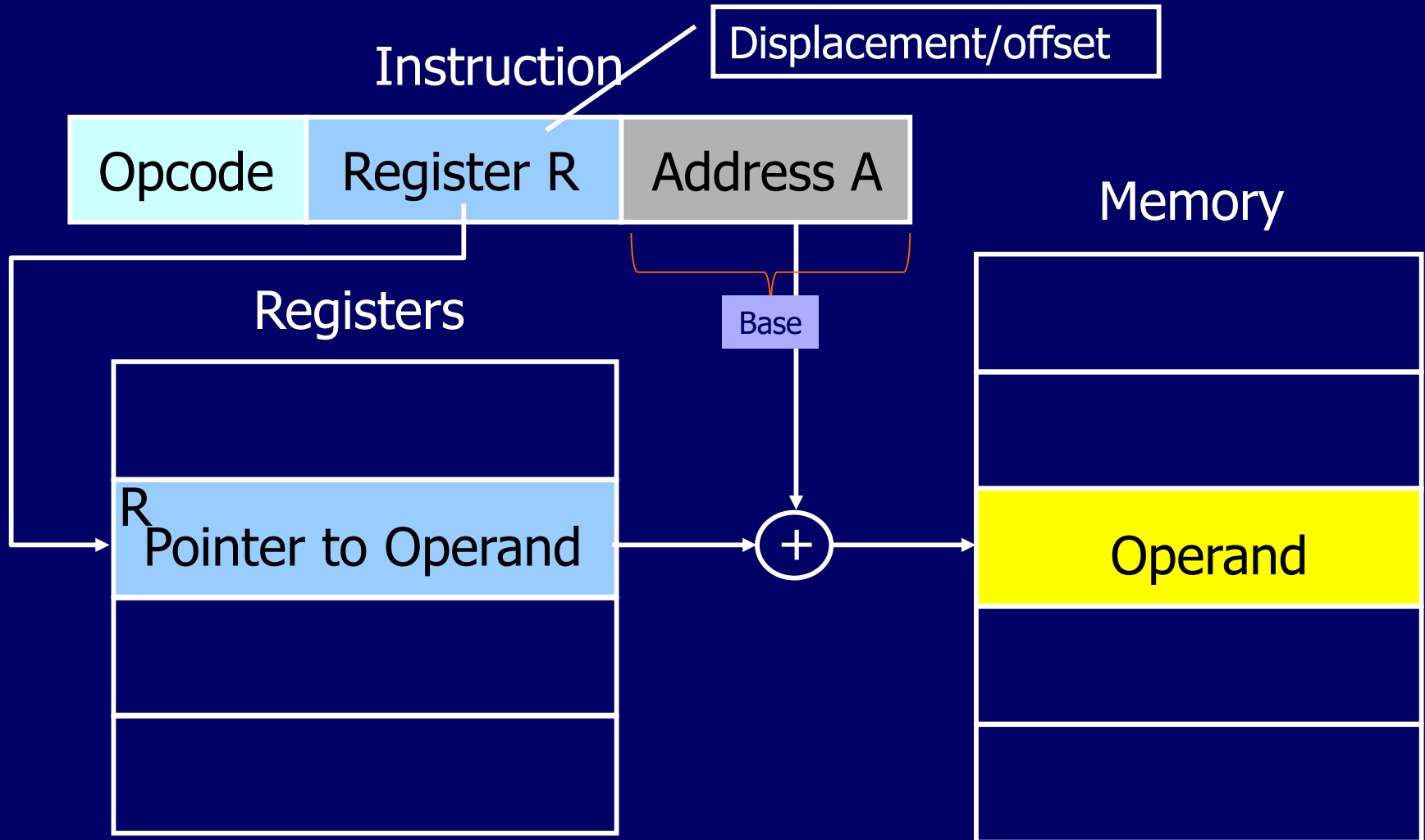
Displacement Addressing

- Combines direct addressing and register indirect addressing
- $EA = A_{[main\ memory]} + (R)_{[register]}$
- Address field hold **two values**
 - $A_{[main\ memory]} = \text{base value (explicit)}$
 - $R_{[main\ memory]} = \text{register that holds displacement, or vice versa}$

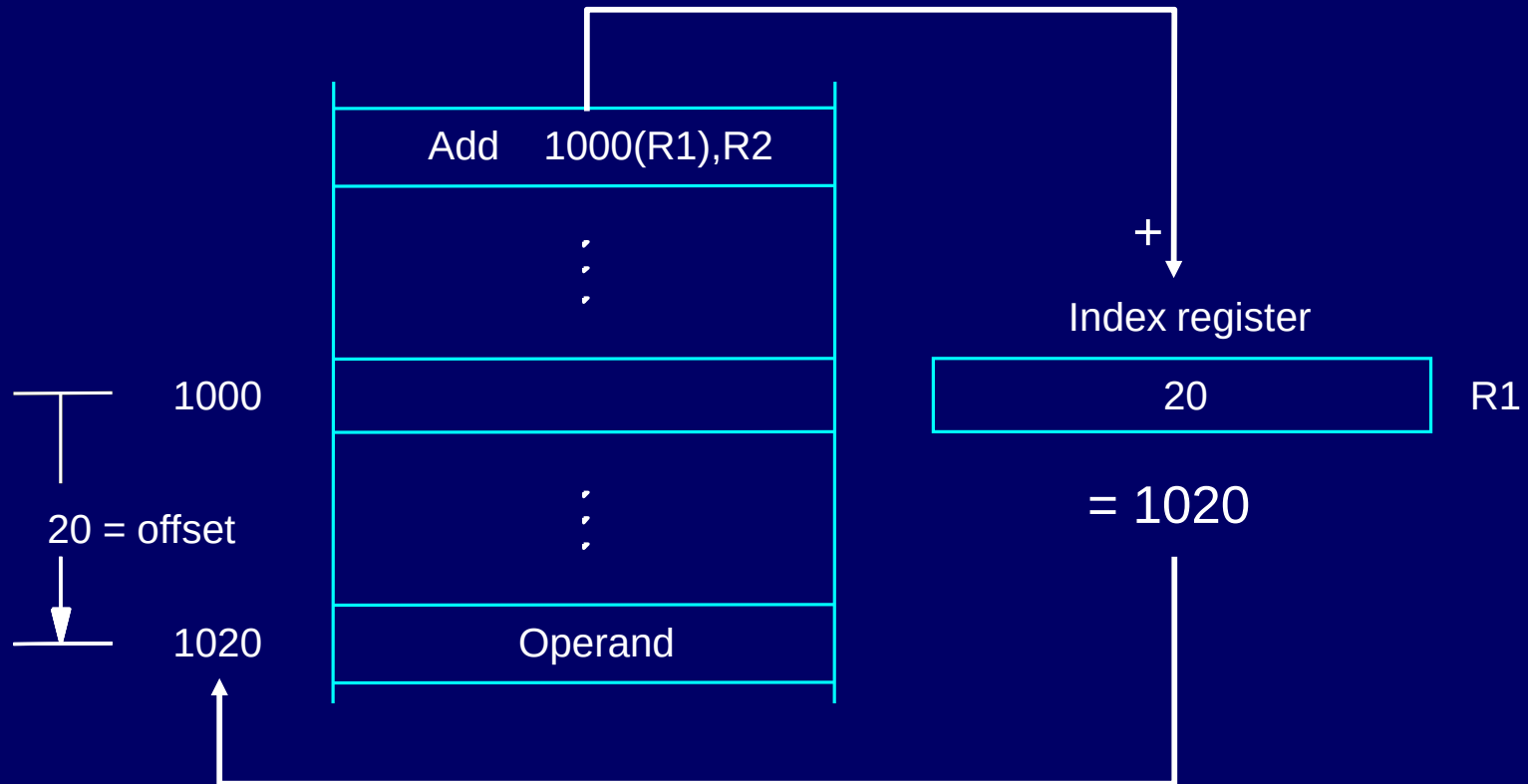
Displacement Addressing

- Common uses of displacement addressing:
 - Relative addressing
 - used in branch instructions (e.g., conditional jumps) where the target address is specified as an offset relative to the current instruction pointer or program counter.
 - Base-register addressing
 - commonly used for accessing elements in arrays or structures. The base register holds the starting address, and the displacement is added to it to access a specific element.
 - Indexing
 - involves multiplying an index by a scale factor and using the result as the displacement. This is particularly useful for accessing elements in arrays.

Displacement Addressing Diagram

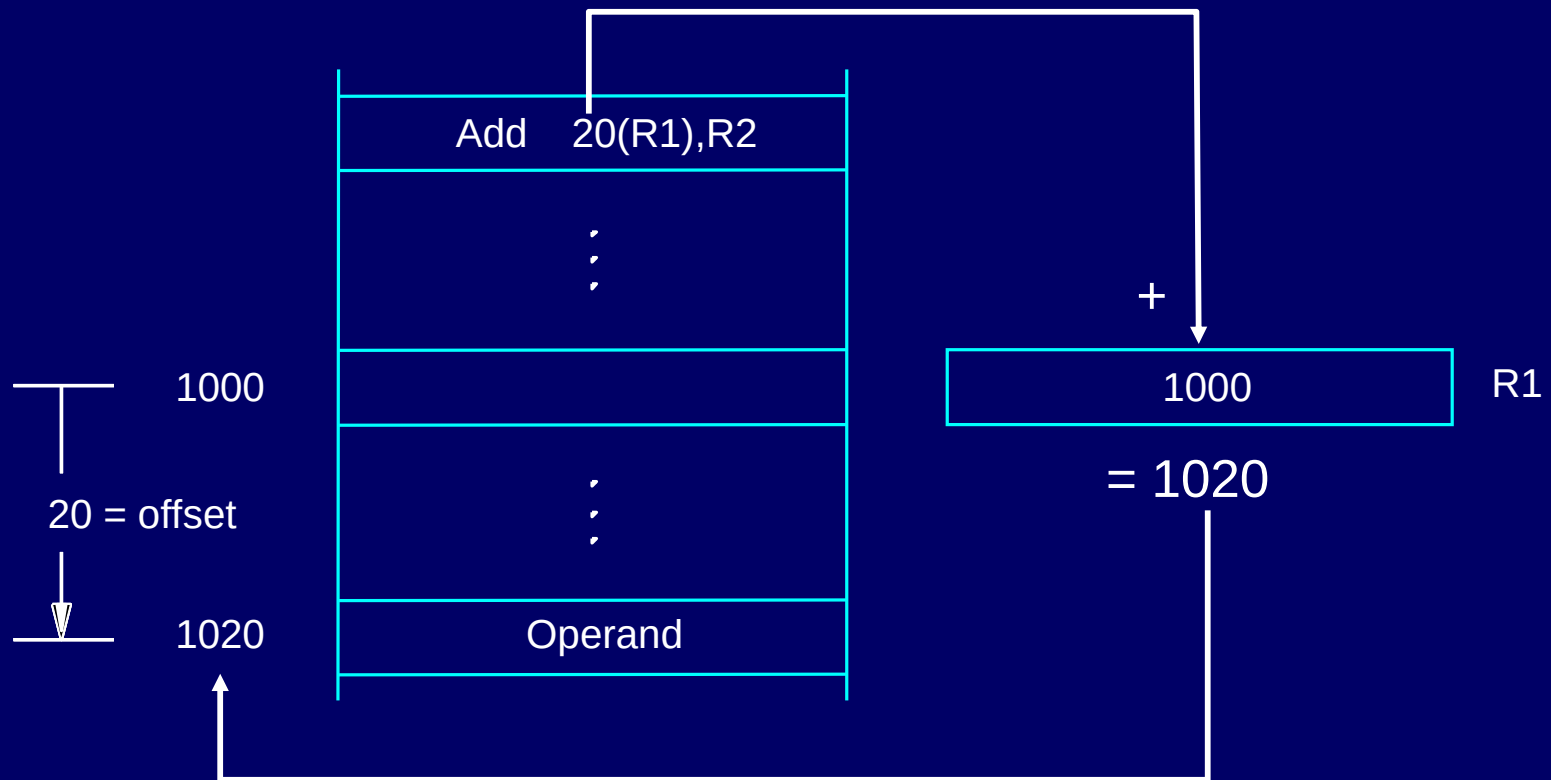


Displacement Addressing Diagram



Offset is in the index register

Displacement Addressing Diagram

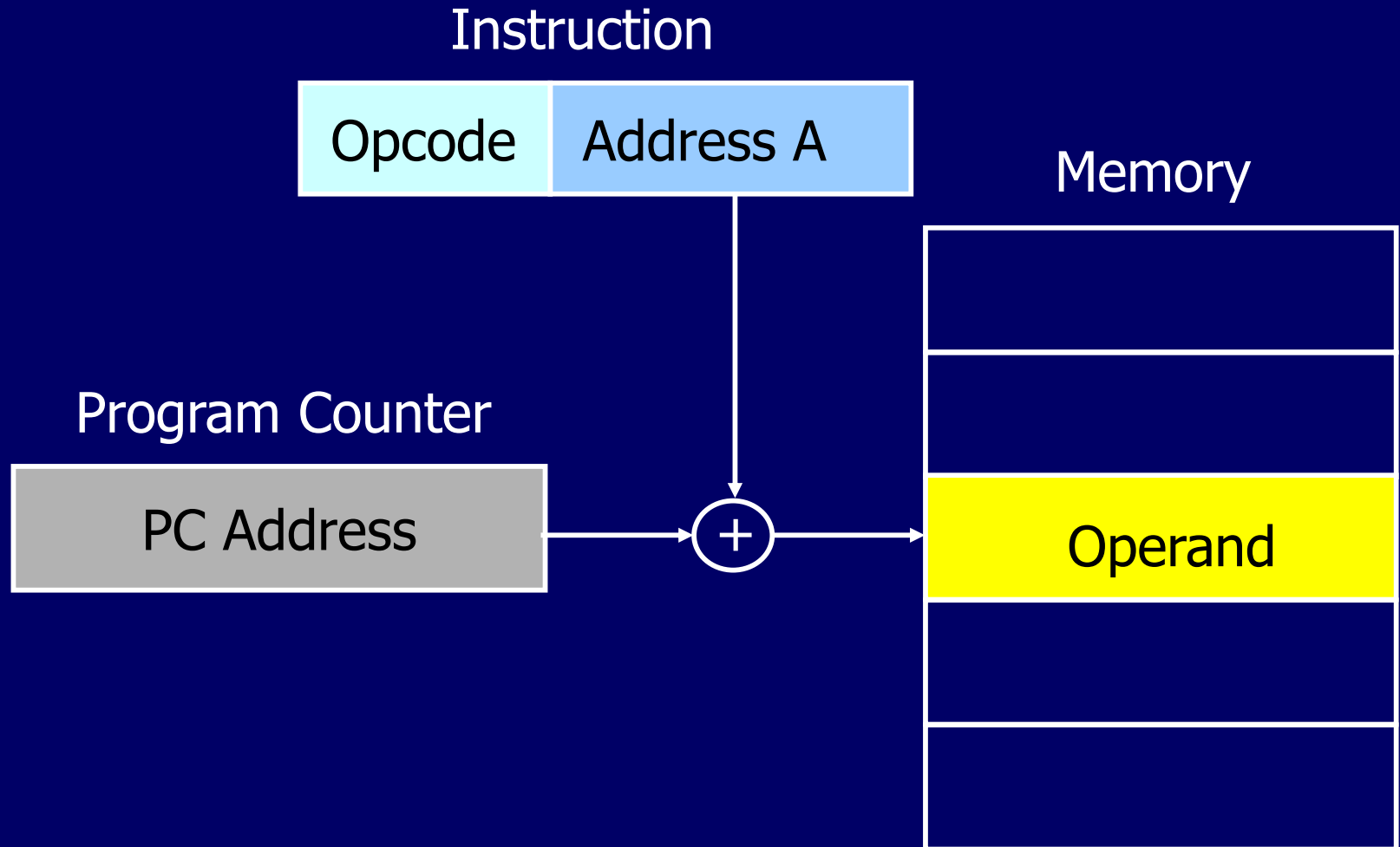


Offset is given as a constant in instruction

Relative Addressing

- A version of displacement addressing
- **R = Program counter, PC**
- $EA = A + (PC)$
 - Current instruction address is added to the address field
- i.e. get operand from A cells from current location pointed to by PC
- c.f concept of locality of reference & cache usage

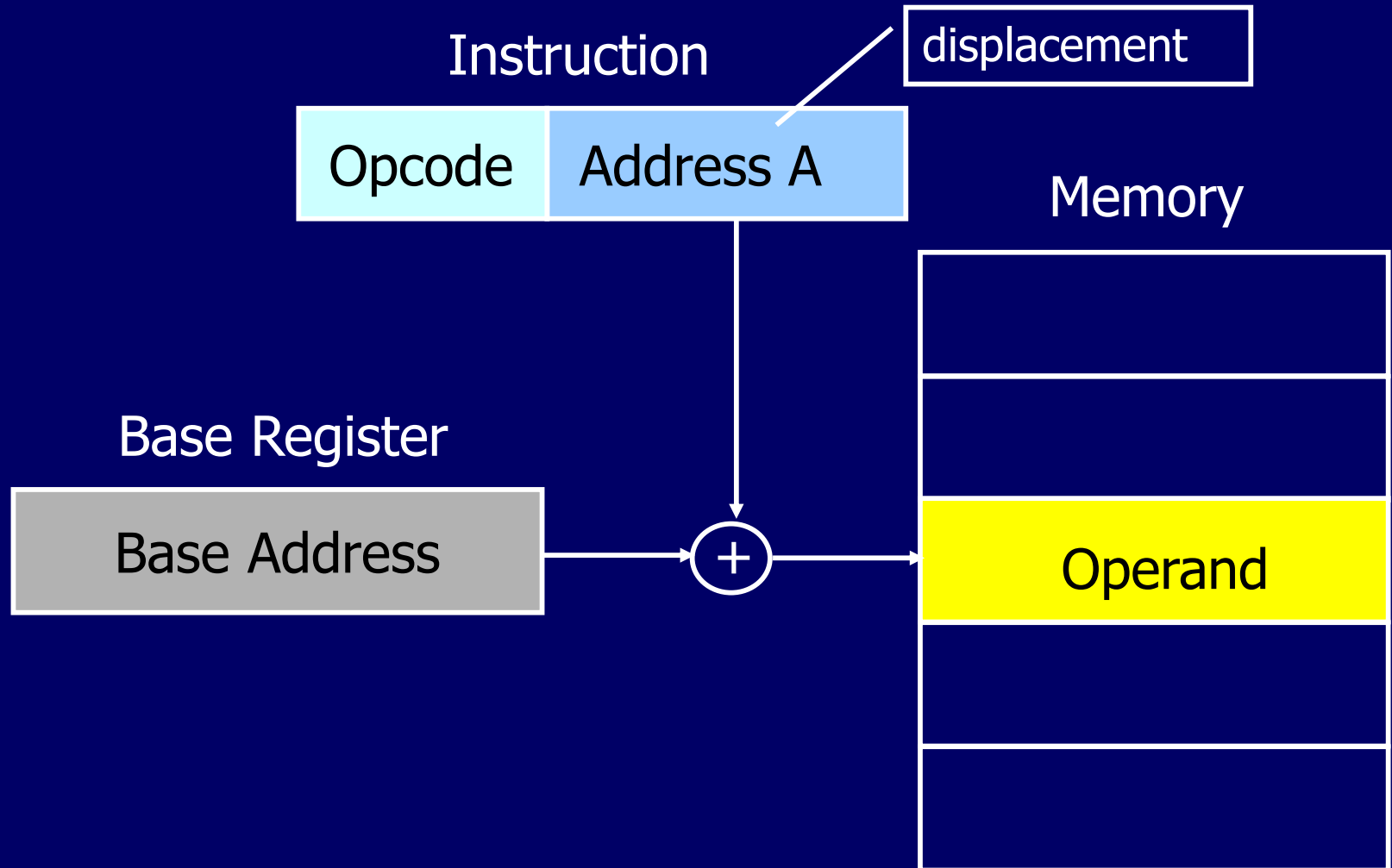
Relative Addressing Diagram



Base-Register Addressing

- A holds displacement
- **R holds pointer to base address**
(of a segment)
- R may be explicit or implicit
- e.g. segment registers in 80x86
- Facilitates relocation of programs in memory
(only the base register requires updating)
- Also exploits the locality of memory references

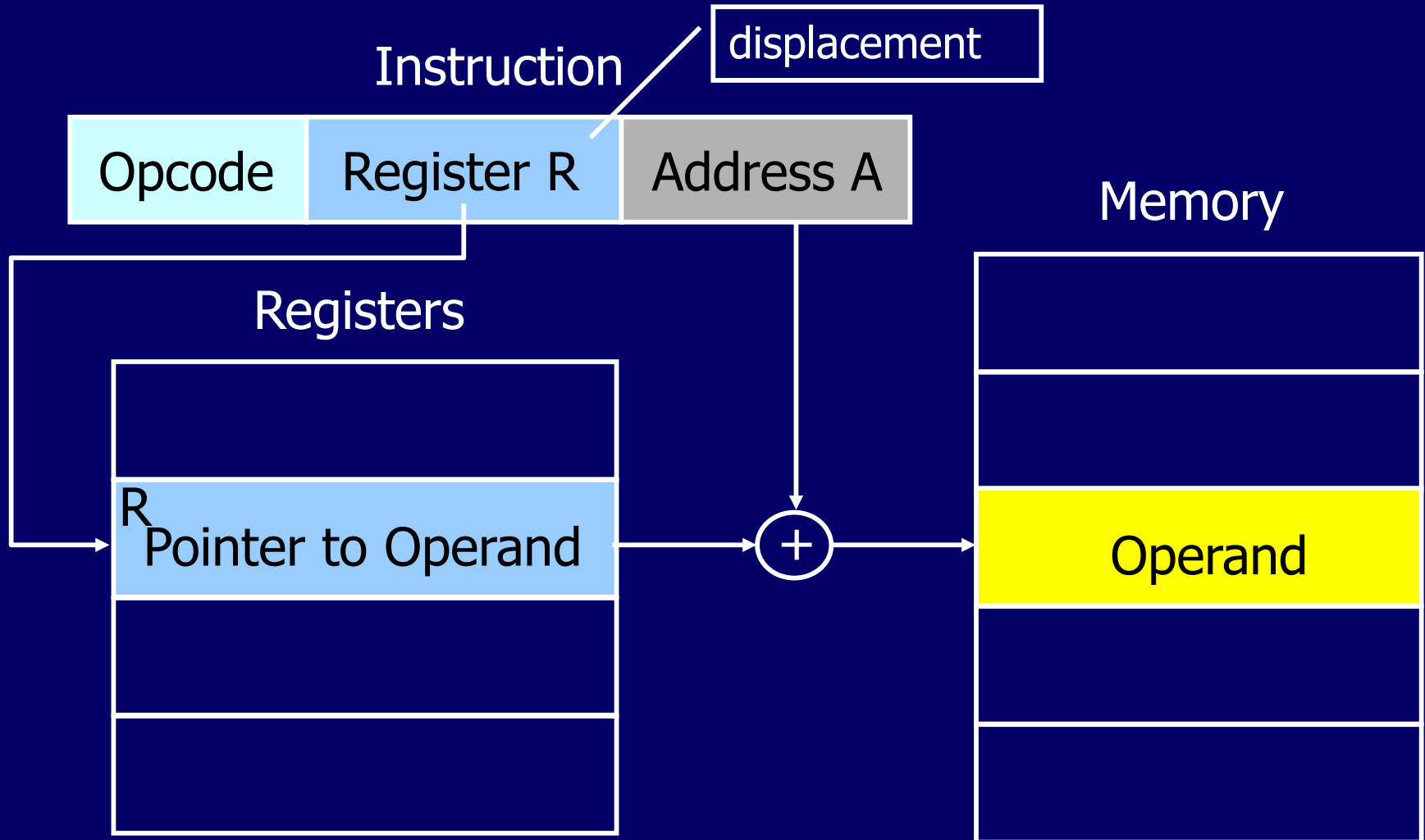
Base-Register Addressing Diagram



Indexed Addressing

- $A = \text{base}$
- $R = \text{displacement}$ (**opposite of base-register addressing**)
- $EA = A + (R)$
- Good for accessing arrays (autoindexing)
 - $EA = A + (R)$
 - $R++$

Indexed Addressing Diagram



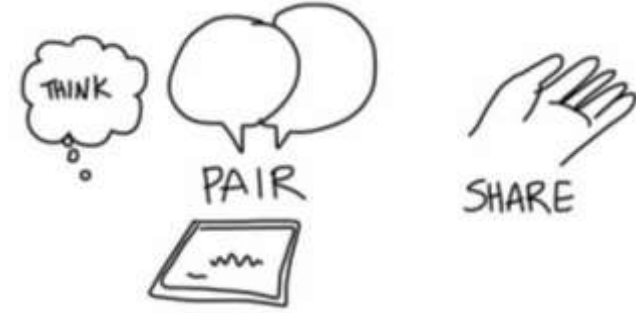
Stack Addressing

- Stack pointer
 - Address of the top of the stack
 - Register
- Register indirect addressing

Comparison

	✓	✗
Immediate	No memory reference	Limited operand magnitude
Direct	Simple	Limited address space
Indirect	Large address space	Multiple memory references
Register	No memory reference	Limited address space
Register indirect	Large address space	Extra memory references
Displacement	Flexibility	Complexity
Stack	No memory reference	Limited applicability

Displacement vs Relative vs Base Register vs Indexed vs Stack



- What's your observation?
- Any similarities/differences?

Outline

- Addressing modes
- Instruction Formats

Instructions Formats

- **Layout of bits** of an instruction
- Must include
 - **Opcode**
 - **Operands** (≥ 0) (implicitly or explicitly)
 - **Addressing mode** (implicitly or explicitly)

Key Issues

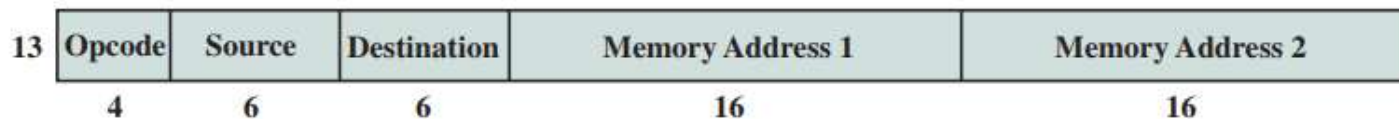
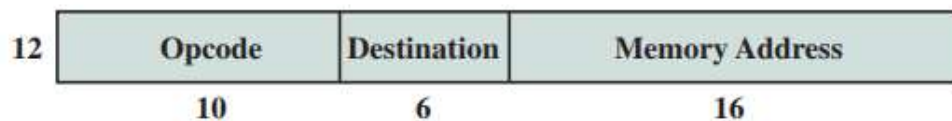
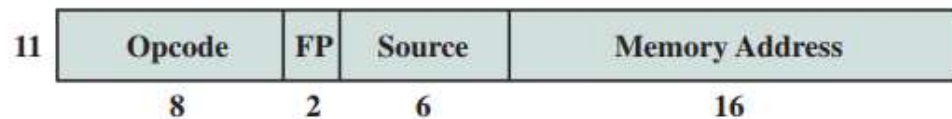
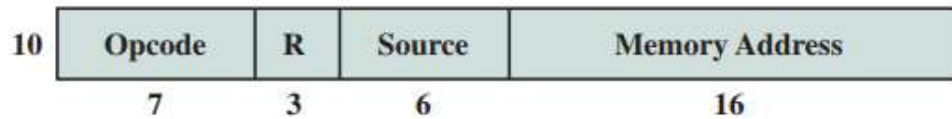
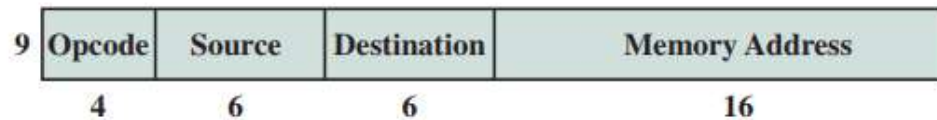
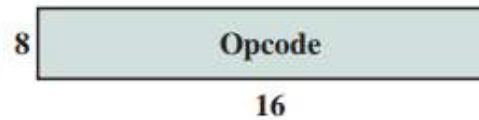
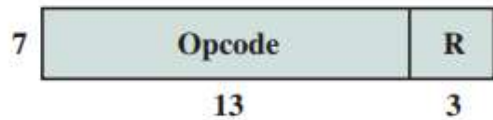
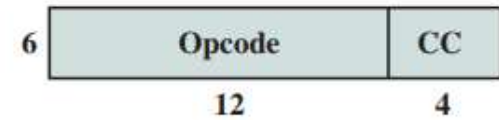
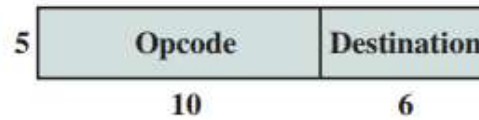
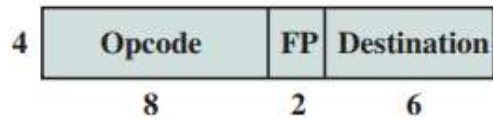
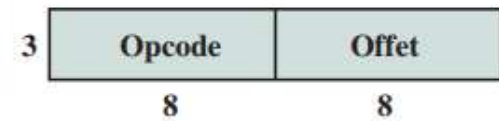
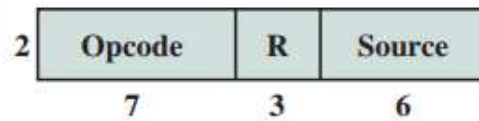
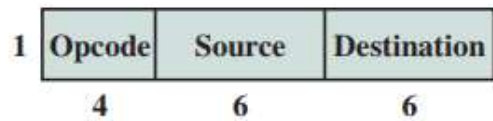
- Instruction **length**
 - Affects (or affected by) memory size, memory organisation, bus structure, processor complexity, and processor speed.
 - Trade-off
 - Powerful instruction vs save space

Key Issues

- Allocation of bits
 - Number of addressing modes
 - Implicit - opcode might always call for indexing
 - Explicit – One or more mode bits are needed
 - Number of operands
 - Typical instructions format – 2 operands
 - Register vs memory
 - Number of register sets
 - Address range
 - Address granularity
- Variable-length instructions

PDP Instructions Formats

- PDP-11
 - 16, 32 & 48 bit instruction lengths
 - 13 different formats (of various lengths and bit allocations)
 - opcodes vary from 4 bits to 16 bits
 - rich set of addressing modes
 - addressing modes depend on operand, not opcode - independence from opcode - orthogonality
 - increase hardware cost and complexity (con)
 - can develop efficient and compact programs (pro)



PDP Instructions Formats

- PDP-11



VAX Instructions Formats

- VAX
 - 2 criteria in designing instruction format
 - all instructions have 'natural' number of operands
 - all operands have same generality in specification
 - highly variable instruction format
 - 1- or 2-byte opcode
 - 0 to 6 operands
 - minimum instruction length = 1 byte
 - maximum instruction length = 37 bytes
 - uses extended opcode with hexadecimal codes

Hexadecimal Format	Explanation	Assembler Notation and Description
8 bits ← 0 5 →	Opcode for RSB	RSB Return from subroutine
D 4 5 9	Opcode for CLRL Register R9	CLRL R9 Clear register R9
B 0 C 4 6 4 0 1 A B 1 9	Opcode for MOVW Word displacement mode, Register R4 356 in hexadecimal Byte displacement mode, Register R11 25 in hexadecimal	MOVW 356(R4), 25(R11) Move a word from address that is 356 plus contents of R4 to address that is 25 plus contents of R11
C 1 0 5 5 0 4 2 D F	Opcode for ADDL3 Short literal 5 Register mode R0 Index prefix R2 Indirect word relative (displacement from PC) Amount of displacement from PC relative to location A	ADDL3 #5, R0, @A[R2] Add 5 to a 32-bit integer in R0 and store the result in location whose address is sum of A and 4 times the contents of R2

VAX Instructions Formats

- VAX



x86 Instruction Formats

- Variety of instruction formats
 - Only opcode is always present
 - Instruction consists of:
 - instruction prefixes (0 to 4) - optional
 - opcode
 - address specifier - optional
 - mod r/m byte
 - scale index byte
 - displacement - optional
 - immediate field - optional
- Prefix bytes
- Instruction fields

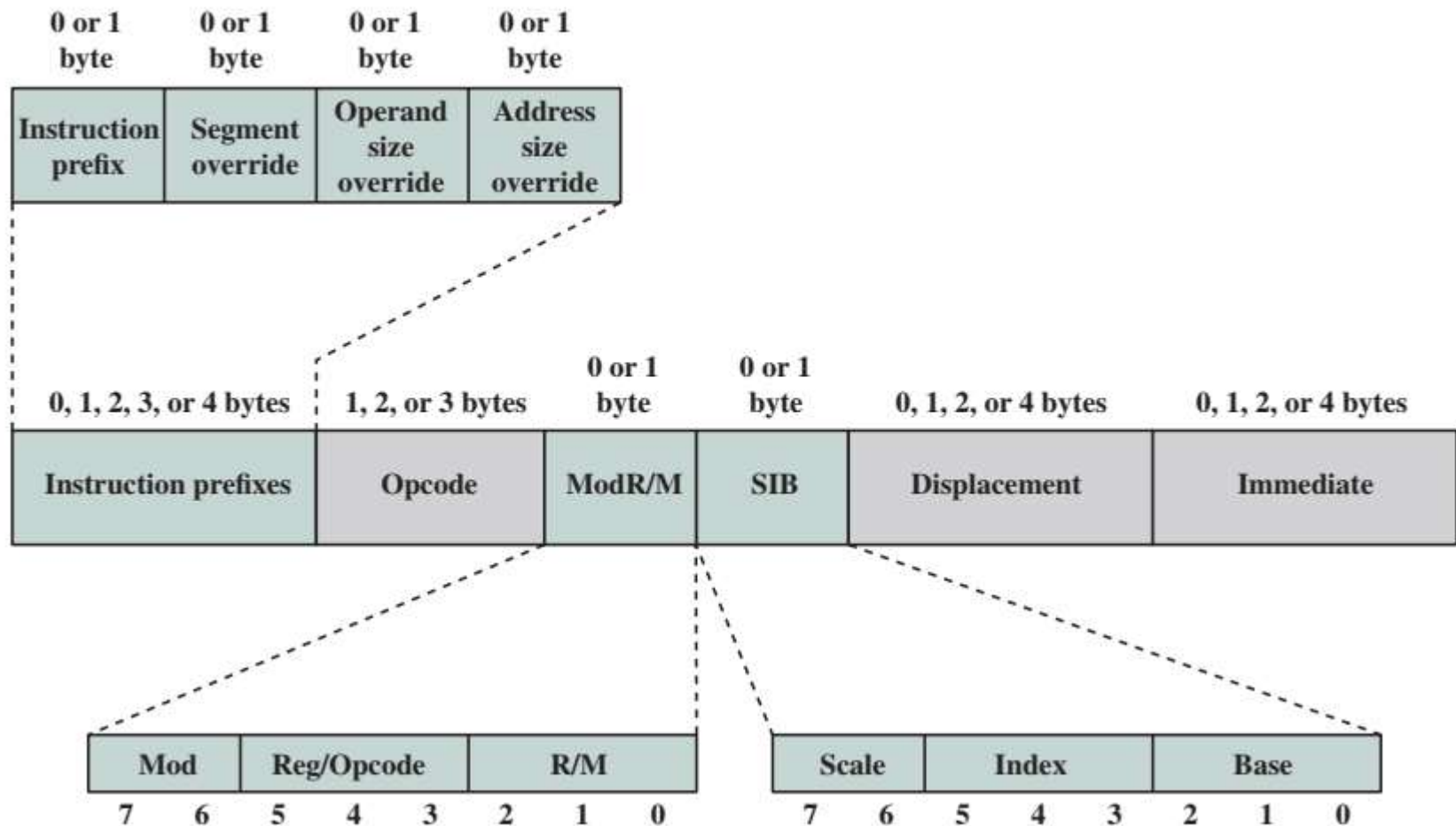
x86 Instruction Formats

- Prefix bytes:
 - Instruction prefixes: LOCK prefix or one of a few repeat prefixes
 - LOCK: to ensure exclusive use of shared memory (multiprocessor environment)
 - Repeat prefixes (REP, REPE, REPZ, REPNE & REPNZ): for string processing
 - Segment override: specifies segment register that an instruction should use (don't use default)
 - Address size: switch between 16- or 32-bit address
 - determines displacement size and offsets during calculation of EA
 - Operand size: switch between 16- or 32-bit operand

x86 Instruction Formats

- Instruction fields:
 - Opcode: 1, 2 or 3 bytes
 - Mod r/m: addressing information
 - specifies if operand is in register or memory
 - if memory, bytes indicate addressing mode
 - Mod r/m byte has 3 fields: Mod field, Reg/Opcode field & r/m field
 - SIB (Scale Index byte): works with Mod r/m
 - SIB has 3 fields: SS field (scaling factor), Index field (index register) & Base field (base register)
 - Displacement: 8-, 16-, or 32-bit signed integer
 - Immediate: 8-, 16-, or 32-bit operand

x86 Instruction Formats



PowerPC Instruction Formats

- All instruction fixed length (32 bits)
- Regular format (similar bit allocation)

e.g. Load/Store



e.g. Branch



PowerPC Instruction Formats

e.g. Arithmetic

